Wootzapp RL & Data Science Intern: 24-Hour Task

Time limit: 24 hours from when you get this. **Work needed:** around 8 to 10 hours.

You may not finish everything. That is okay. Do the main parts properly, and tell us what you skipped and why.

After you submit, we will have a live call. On that call you will make changes to your own code while we watch. So make sure you understand every line you write.

What you have to build

You will build three things:

1. A small shopping web page.
2. A program that opens this page in Chrome, clicks buttons on it, and gives a score. In RL this is called the **environment**.
3. An **agent** that learns, by trial and error, how to buy the correct item on the page.

At the end, you will study your results and write a short report.

Part 1: The web page (about 1 hour)

Make one file: `site/index.html`. Use only plain HTML and JavaScript. Do not use React or any other framework.

The page is a small shop called MiniShop. It has 4 screens:

Screen	What is on it
Catalog	4 products, each with a "View" button. Also one extra button "Newsletter". It opens a useless screen that has only a "Back" button.
Product	"−" and "+" buttons to set quantity (1 to 3), "Add to cart" (opens the Cart screen), and "Back".
Cart	Shows the item and quantity. Buttons: "Checkout", "Clear cart", "Back".
Done	Shows "Order placed" with the item and quantity.

The goal comes from the URL. Example:

`index.html?item=blue-mug&qty=2&seed=42&popup_p=0.15&delay_p=0.10`

This means: the agent should buy 2 blue mugs.

The page must misbehave sometimes, like real websites do:

- Each time the screen changes, a popup may appear and cover the page. Nothing else can be clicked until "Dismiss" is clicked. The chance of this is `popup_p` (0.15 means 15%).
- Each time the screen changes, the buttons may appear late, after 50 to 300 ms. The chance of this is `delay_p`.
- Use the `seed` value for the random numbers, so the same seed always gives the same behaviour.

Success means the order is placed with exactly the right item and the right quantity. Anything else is a failure.

Part 2: The environment, in Rust or C++ (main part)

Write a program in **Rust or C++** that does the following.

Start Chrome and control it. Start Chromium in headless mode (no window). Talk to it using the Chrome DevTools Protocol (CDP). CDP is simple: you send JSON messages over a WebSocket, and Chrome sends JSON replies.

- Allowed: libraries for WebSocket, JSON, async, and command-line arguments.
- Not allowed: ready-made browser control libraries such as Puppeteer, Playwright, Selenium, chromiumoxide, or headless_chrome. We want to see you work with CDP yourself.

Give two functions.

- `reset(task, seed)`: opens the page fresh for a new attempt and returns what the agent can see.
- `step(action)`: does one action and returns five things: what the agent sees now, the reward, whether the attempt ended, whether it was stopped for taking too long, and any extra info.

What the agent sees (observation). The current screen name, the goal, and a list of buttons that are visible right now. For each button give its number, its text, and whether it can be clicked. Read this from the real page every time. Do not hard-code the button list for each screen.

Example:

```json
{"screen": "product", "goal": "blue-mug x2",
 "buttons": [{"i": 0, "text": "-"}, {"i": 1, "text": "+"},
             {"i": 2, "text": "Add to cart"}, {"i": 3, "text": "Back"}]}
```

What the agent can do (actions). Only two: `click(i)` to click button number i, and `wait` to do nothing for a moment. The click must be a real mouse click sent through CDP at the button's position on the page. Do not call `.click()` from JavaScript.

Reward. You decide the reward numbers. The only truth is: was the correct order placed or not. If you give small rewards along the way, explain why.

It must not get stuck. Add a time limit for each step. Stop any attempt after 20 steps. When your program ends, no Chrome process should be left running.

Save a log. For every step, write one line of JSON to a file. Include: attempt number, seed, goal, step number, what the agent saw, action, reward, whether it ended, time taken in ms, and whether a popup was showing.

Example line:

```json
{"episode": 17, "seed": 42, "goal": "blue-mug x2", "step": 3, "action": "click(1)", "reward": 0, "done": false
```

Part 3: The agents (main part)

Build two agents:

1. **Random agent.** It clicks any button at random. This is your baseline to compare against.

2. **Learning agent.** It gets better with practice. Simple Q-learning with a table is enough. You do not need a GPU or neural networks.

You can write the agents in Rust/C++, or in Python if you connect Python to your environment in some way.

Rules:

- Train on all 12 goals (4 items × 3 quantities).
- Repeat training at least 3 times with different seeds.
- Training and testing together must finish in under 20 minutes on a laptop with 8 GB RAM.

Part 4: Study your results (main part)

Write a Python notebook or script that reads your log files and creates `report.md` with charts. Answer these questions:

1. How often does the random agent succeed? How often does the learning agent succeed? Test each on at least 200 attempts. Also show the error range (confidence interval).
2. Show a chart of how the learning agent improves during training. Show the average of your 3 runs and how much they differ.
3. Test the learning agent with `popup_p` set to 0, 0.15 and 0.4. What changes? Why?
4. When the learning agent fails, what are the 3 most common reasons? Show examples from your logs.
5. How long does one step take? Give the typical time (median) and the slow case (95th percentile). What takes the most time?
6. Tell us one thing that surprised you.

Every number in your report must come from your logs. We will run your code again and check.

Part 5: Extra (optional, pick only one)

- Run several environments at the same time on one Chrome, and report how much faster it gets.
- Package one goal as a task in the Harbor format.

What to submit

Send us two links: your GitHub repo and your Loom video.

Put your code in a **public** GitHub repo. Commit your work as you go. Do not push everything in one commit at the end.

The repo must have:

- `run.sh`: one command that builds the code, trains, tests, and creates the report. It must work on Linux or WSL2 with Chromium installed.
- `README.md`: how to set up and run.
- `DECISIONS.md` (one page or less): what your state, actions and reward are, when an attempt ends, what you skipped, and what you would do next if you had more time.
- **A Loom video** of 5 to 10 minutes. Share your screen and explain everything you built:
  - Show the web page and how the popup and delay work.
  - Run `run.sh` and show the agent learning.
  - Walk through your main code files and explain what each one does.
  - Show your report and explain your results.
  - Tell us what you skipped and what was hardest.

How we will judge

- Does it run when we download it fresh?
- Is the code clean, and does it handle problems without crashing?
- Is the RL part correct?
- Is the report honest and backed by data?
- Is everything explained clearly?

A small project that works is better than a big one that is broken.